

MODULE 12

Machine Learning

Topics Covered

Lectures 1–3 — AI/ML Foundations & Types of ML
Lectures 4–5 — Tensors & ML Development Life Cycle
Lectures 6–8 — Demo ML Models
Lectures 9–13 — Business Problem Framing & Data Gathering
Lectures 14–16 — Data Cleaning
Lectures 17–19 — EDA & Feature Engineering
Lectures 20–24 — Feature Encoding, Scaling & ML Pipeline
Lectures 25–28 — Core ML Algorithms
Lectures 29–31 — Evaluation Metrics
Lectures 32–35 — Advanced Algorithms & Validation
Lectures 36–40 — Unsupervised Learning & Clustering

Lectures 1–3: AI/ML Foundations & Types of ML

What is AI and Machine Learning?

Artificial Intelligence (AI) is the broad science of building machines that can perform tasks that normally require human intelligence. Machine Learning (ML) is a subset of AI where systems learn from data rather than being explicitly programmed.

Term	Definition
Artificial Intelligence	Broad field — any technique that enables machines to mimic human behaviour
Machine Learning	Subset of AI — systems learn patterns from data to make predictions
Deep Learning	Subset of ML — uses neural networks with many layers
NLP	Natural Language Processing — AI for understanding/generating text
Generative AI	AI that creates new content (text, images, audio) — e.g. ChatGPT, DALL·E

□ Hierarchy

GenAI $\subset$ DL $\subset$ ML $\subset$ AI. Every deep learning model is an ML model; every ML model is AI — but not vice versa.

Types of Machine Learning

Type	Description Example
Supervised Learning	Train on labelled data (input $\rightarrow$ output pairs). Predict output for new inputs. Ex: spam detection, house price prediction
Unsupervised Learning	Find hidden patterns in unlabelled data. No predefined output. Ex: customer segmentation, anomaly detection
Semi-Supervised	Small amount of labelled + large unlabelled data. Ex: image classification with few labels
Reinforcement Learning	Agent learns by taking actions and receiving rewards/penalties. Ex: chess AI, robotics

Supervised Learning Sub-types

Sub-type	Output Examples
Regression	Continuous number $\rightarrow$ house price, temperature, stock value
Classification	Discrete category $\rightarrow$ spam/not spam, dog/cat, disease/no disease

Lectures 4–5: Tensors & ML Development Life Cycle

Tensors in Machine Learning

A tensor is a generalised mathematical object — a container for numerical data. All ML data is eventually represented as tensors.

Rank	Name Example
0D (scalar)	Single number → 3.14, 42
1D (vector)	Array → [1, 2, 3, 4]
2D (matrix)	Table/spreadsheet → shape (rows, cols)
3D tensor	Stack of matrices → e.g. image (height, width, channels)
4D tensor	Batch of images → (batch, height, width, channels)

```

import numpy as np
import torch

scalar = torch.tensor(5.0)           # 0D
vector = torch.tensor([1.0, 2.0, 3.0]) # 1D
matrix = torch.tensor([[1,2],[3,4],[5,6]]) # 2D shape (3,2)
image = torch.zeros(3, 224, 224)      # 3D: RGB image
batch = torch.zeros(32, 3, 224, 224)  # 4D: batch of 32

print(matrix.shape)    # torch.Size([3, 2])
print(matrix.dtype)    # torch.int64

```

ML Development Life Cycle (MLDLC)

Phase	Key Activities
1. Problem Framing	Define business objective → translate to ML problem (classification? regression?)
2. Data Gathering	CSV, APIs, SQL databases, web scraping, Kaggle datasets
3. Data Cleaning	Handle missing values, remove duplicates, fix formats, treat outliers
4. EDA	Explore distributions, correlations, patterns — understand the data
5. Feature Engineering	Create/transform features to give the model better signals
6. Model Building	Choose algorithm, train model, tune hyperparameters
7. Evaluation	RMSE, MAE, Accuracy, Precision, Recall, F1, ROC-AUC
8. Deployment	Serve via API (Flask/FastAPI), monitor performance, retrain periodically

Lectures 6–8: Building Demo ML Models

These lectures walk through end-to-end ML model building. The pattern is the same regardless of algorithm — learn this workflow once and apply it everywhere.

Standard ML Workflow in scikit-learn

```

import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

# 1. Load data
df = pd.read_csv('data.csv')

# 2. Features (X) and target (y)
X = df.drop('target', axis=1)

```

```

y = df['target']

# 3. Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# 4. Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)    # fit + transform
X_test = scaler.transform(X_test)         # transform ONLY

# 5. Train model
model = LinearRegression()
model.fit(X_train, y_train)

# 6. Predict & evaluate
y_pred = model.predict(X_test)
print('RMSE:', np.sqrt(mean_squared_error(y_test, y_pred)))
print('R²: ', r2_score(y_test, y_pred))

```

□ Key rule — fit on train, transform on test

Always call `.fit_transform()` on training data.

Only call `.transform()` on test/validation data.

Fitting on test data causes data leakage — your model 'peeks' at test info during training.

Lectures 9–13: Business Problem Framing & Data Gathering

Framing the Business Problem

Before writing a single line of code, translate the business objective into a specific ML task:

Business Question	ML Formulation
Will this customer churn?	Binary classification (churn / no churn)
What will next month's revenue be?	Regression (predict continuous value)
Which customer segment is this?	Clustering (unsupervised)
Is this transaction fraudulent?	Anomaly detection / Binary classification
What product should I recommend?	Recommendation system

Data Gathering Methods

Source	How to Load
CSV file	<code>pd.read_csv('file.csv')</code>
JSON file	<code>pd.read_json('file.json')</code> or <code>json.load(f)</code>
SQL database	<code>pd.read_sql(query, connection)</code> via <code>sqlalchemy</code>
REST API	<code>requests.get(url, params={...}).json()</code>
Kaggle dataset	kaggle datasets download -d owner/name
Web scraping	BeautifulSoup + requests or Scrapy
<pre> # Fetching data from a REST API import requests, pandas as pd </pre>	

Source	How to Load
	<pre> url = 'https://api.example.com/data' params = {'start': '2024-01-01', 'limit': 1000} headers= {'Authorization': 'Bearer YOUR_API_KEY'} response = requests.get(url, params=params, headers=headers) data = response.json() df = pd.DataFrame(data['records']) # SQL with SQLAlchemy from sqlalchemy import create_engine engine = create_engine('postgresql://user:pass@host/db') df = pd.read_sql('SELECT * FROM sales WHERE year=2024', engine) </pre>

Lectures 14–16: Data Cleaning

Real-world data is messy. Data cleaning (also called data wrangling) is typically 60–80% of an ML project's effort.

Types of Issues & Fixes

Problem	Solution
Missing values	dropna() / fillna(mean) / fillna(ffill) / imputation
Duplicate rows	df.drop_duplicates()
Wrong data types	df[col].astype('float')
Inconsistent casing	df[col].str.lower().str.strip()
Outliers	IQR method, Z-score, or domain-driven capping
Inconsistent labels	df[col].replace({'Yes':1, 'No':0})
Extra whitespace	df[col].str.strip()
<pre> # Outlier removal using IQR Q1 = df['Salary'].quantile(0.25) Q3 = df['Salary'].quantile(0.75) IQR = Q3 - Q1 lower = Q1 - 1.5 * IQR upper = Q3 + 1.5 * IQR df_clean = df[(df['Salary'] >= lower) & (df['Salary'] <= upper)] # Automated cleaning pipeline pattern def clean_dataframe(df): df = df.drop_duplicates() df = df.dropna(thresh=int(len(df.columns) * 0.6)) # drop cols >40% NaN for col in df.select_dtypes(include='object').columns: df[col] = df[col].str.strip().str.lower() df = df.fillna(df.median(numeric_only=True)) return df </pre>	

□ Importance in LangChain Apps

When building RAG pipelines or LLM apps, garbage data → garbage embeddings → poor retrieval.

Clean, consistent text chunks dramatically improve LLM response quality.

Lectures 17–19: EDA & Feature Engineering

Exploratory Data Analysis (EDA)

EDA is the process of visually and statistically exploring data to understand its structure, distributions, and relationships before modelling.

EDA Step	Tools / Code
Shape & types	<code>df.shape, df.dtypes, df.info()</code>
Missing values	<code>df.isnull().sum(), sns.heatmap(df.isnull())</code>
Distributions	<code>df.describe(), df.hist(), sns.boxplot()</code>
Correlations	<code>df.corr(), sns.heatmap(df.corr(), annot=True)</code>
Categorical counts	<code>df['col'].value_counts(), sns.countplot()</code>
Outlier visualisation	<code>sns.boxplot(df['col'])</code>
Pairwise scatter	<code>sns.pairplot(df)</code>

Feature Engineering

Feature engineering creates new variables from existing ones to give the model stronger predictive signals.

Technique	Example
Extract from datetime	<code>df['year'] = df['date'].dt.year; df['month'] = df['date'].dt.month</code>
Binning	<code>pd.cut(df['age'], bins=[0,18,35,60,100], labels=['teen','adult','mid','senior'])</code>
Log transform	<code>df['log_salary'] = np.log1p(df['salary'])</code> — normalises skewed data
Interaction feature	<code>df['price_per_sqft'] = df['price'] / df['area']</code>
Polynomial features	<code>from sklearn.preprocessing import PolynomialFeatures</code>
Aggregation	<code>df.groupby('dept')['salary'].transform('mean')</code> — group mean as feature

```

# Log transformation for skewed feature
import numpy as np
df['log_income'] = np.log1p(df['income']) # log1p handles zeros

# Binning ages
df['age_group'] = pd.cut(df['age'],
    bins=[0, 18, 35, 60, 100],
    labels=['Teen', 'Young Adult', 'Mid', 'Senior'])

# Extract datetime features
df['date'] = pd.to_datetime(df['date'])
df['day_of_week'] = df['date'].dt.dayofweek
df['is_weekend'] = df['day_of_week'].isin([5, 6]).astype(int)

```

Lectures 20–24: Feature Encoding, Scaling & ML Pipeline

Feature Encoding

ML algorithms require numerical inputs. Categorical variables must be encoded before training.

Encoding Method	When to Use Example
Label Encoding	Ordinal categories with natural order: Small < Medium < Large

Encoding Method	When to Use Example
One-Hot Encoding	Nominal categories (no order): City, Colour, Gender
Ordinal Encoding	Custom ordered categories — manually specify order
Target Encoding	High-cardinality categoricals — replace with target mean (use carefully)
Binary Encoding	Many categories — encodes as binary bits to reduce dimensions

```

from sklearn.preprocessing import LabelEncoder, OneHotEncoder
from sklearn.compose import ColumnTransformer

# Label Encoding
le = LabelEncoder()
df['size_enc'] = le.fit_transform(df['size']) # Small→0, Med→1, Large→2

# One-Hot Encoding with pandas
df = pd.get_dummies(df, columns=['city', 'gender'], drop_first=True)

# One-Hot with sklearn
ohe = OneHotEncoder(sparse_output=False, drop='first')
encoded = ohe.fit_transform(df[['city']])

```

Column Transformer

```

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

num_features = ['age', 'salary', 'experience']
cat_features = ['city', 'department']

preprocessor = ColumnTransformer(transformers=[
    ('num', StandardScaler(), num_features),
    ('cat', OneHotEncoder(drop='first'), cat_features),
])

X_transformed = preprocessor.fit_transform(X_train)

```

Feature Scaling

Method	Formula When to Use
Standardisation (Z-score)	$z = (x - \mu) / \sigma \rightarrow \text{mean}=0, \text{std}=1$. Use when data is roughly normal. Works with outliers.
Min-Max Normalisation	$x' = (x - \min) / (\max - \min) \rightarrow \text{scales to } [0,1]$. Use when bounded range is needed. Sensitive to outliers.
Robust Scaler	Uses median & IQR instead of mean & std. Best when data has outliers.

```

from sklearn.preprocessing import StandardScaler, MinMaxScaler, RobustScaler

ss = StandardScaler()
X_std = ss.fit_transform(X_train)

mm = MinMaxScaler()
X_norm = mm.fit_transform(X_train)

```

ML Pipeline

A Pipeline chains preprocessing steps and a model into one object — preventing data leakage and simplifying code.

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = Pipeline([
    ('scaler', StandardScaler()),
    ('model', LogisticRegression()),
])

pipe.fit(X_train, y_train) # fit scaler + model in one call
y_pred = pipe.predict(X_test) # automatically applies scaling

# With ColumnTransformer
full_pipe = Pipeline([
    ('preprocess', preprocessor), # ColumnTransformer
    ('model', RandomForestClassifier()),
])
```

Lectures 25–28: Core ML Algorithms

Linear Regression

Models a continuous target as a weighted sum of input features. Best for linear relationships.

Key Concept	Details
Equation	$\hat{y} = w_0 + w_1 x_1 + w_2 x_2 + \dots + w_n x_n$
Cost function	$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$ — minimised via gradient descent
Assumptions	Linearity, independence, homoscedasticity, normality of residuals
Regularisation	Ridge (L2) and Lasso (L1) penalise large weights to prevent overfitting

```
from sklearn.linear_model import LinearRegression, Ridge, Lasso

lr = LinearRegression()
ridge = Ridge(alpha=1.0) # L2 regularisation
lasso = Lasso(alpha=0.1) # L1 regularisation (also does feature selection)

lr.fit(X_train, y_train)
print('Coefficients:', lr.coef_)
print('Intercept: ', lr.intercept_)
```

Logistic Regression

Despite the name, it is a CLASSIFICATION algorithm. Uses the sigmoid function to output probabilities between 0 and 1.

Key Concept	Details
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z}) \rightarrow$ maps any value to (0,1)
Decision	Predict class 1 if $P \geq 0.5$, else class 0 (threshold is adjustable)
Multi-class	Use <code>multi_class='ovr'</code> (one vs rest) or 'multinomial'
Cost function	Log-loss (binary cross entropy) — not MSE

```
from sklearn.linear_model import LogisticRegression
```


Key Concept	Details
	<pre> model = LogisticRegression(C=1.0, max_iter=200) model.fit(X_train, y_train) probs = model.predict_proba(X_test)[:, 1] # P(class=1) preds = model.predict(X_test) </pre>

Naive Bayes

Probabilistic classifier based on Bayes' theorem, assuming all features are independent (the 'naive' assumption). Very fast and works well for text.

Variant	Best For
GaussianNB	Continuous features (assumes normal distribution)
MultinomialNB	Count/frequency features — document classification, NLP
BernoulliNB	Binary features (0/1)
<pre> from sklearn.naive_bayes import GaussianNB, MultinomialNB gnb = GaussianNB() gnb.fit(X_train, y_train) print(gnb.score(X_test, y_test)) </pre>	

K-Nearest Neighbors (KNN)

Classifies a new point based on the majority class of its K nearest neighbours. No training phase — it memorises the data ('lazy learner').

Key Concept	Details
Distance metric	Euclidean (default), Manhattan, Minkowski
Choosing K	Small K → complex boundary, overfitting. Large K → smooth, underfitting. Use cross-validation.
Scaling	REQUIRED — KNN is distance-based; unscaled features dominate
Time complexity	O(n) per prediction — slow on large datasets
<pre> from sklearn.neighbors import KNeighborsClassifier knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean') knn.fit(X_train, y_train) print(knn.score(X_test, y_test)) # Find best K using cross-validation from sklearn.model_selection import cross_val_score scores = [cross_val_score(KNeighborsClassifier(k), X, y, cv=5).mean() for k in range(1, 21)] best_k = scores.index(max(scores)) + 1 </pre>	

Lectures 29–31: Evaluation Metrics

Regression Metrics

Metric	Formula Interpretation
MAE (Mean Absolute Error)	$MAE = (1/n) \sum y_i - \hat{y}_i $ — average absolute error. Same units as target. Robust to outliers.
MSE (Mean Squared Error)	$MSE = (1/n) \sum (y_i - \hat{y}_i)^2$ — squares errors, penalises large errors heavily.
RMSE (Root MSE)	$RMSE = \sqrt{MSE}$ — same units as target. Most interpretable. Most common.
R^2 (Coefficient of determination)	$R^2 = 1 - SS_{res}/SS_{tot}$. Range: 0–1. 1 = perfect fit. Can be negative for bad models.

```

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

mae = mean_absolute_error(y_test, y_pred)
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f'MAE: {mae:.2f}')
print(f'RMSE: {rmse:.2f}')
print(f'R²: {r2:.4f}')

```

Classification Metrics

The Confusion Matrix is the foundation of all classification metrics:

Term	Meaning
TP (True Positive)	Predicted Positive, Actually Positive — correct
TN (True Negative)	Predicted Negative, Actually Negative — correct
FP (False Positive)	Predicted Positive, Actually Negative — Type I error
FN (False Negative)	Predicted Negative, Actually Positive — Type II error
Accuracy	$(TP+TN) / \text{Total}$ — good when classes are balanced
Precision	$TP / (TP+FP)$ — of all predicted positive, how many are truly positive?
Recall (Sensitivity)	$TP / (TP+FN)$ — of all actual positives, how many did we catch?
F1 Score	$2 \times (P \times R) / (P + R)$ — harmonic mean. Best for imbalanced classes.

```

from sklearn.metrics import (accuracy_score, precision_score,
                             recall_score, f1_score, confusion_matrix,
                             classification_report)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
print('Accuracy: ', accuracy_score(y_test, y_pred))
print('Precision:', precision_score(y_test, y_pred))
print('Recall:   ', recall_score(y_test, y_pred))
print('F1:      ', f1_score(y_test, y_pred))

```

□ Precision vs Recall trade-off

High Precision needed: spam filter (don't want legit emails flagged).

High Recall needed: cancer detection (don't want to miss a real case).

Use F1 when you need balance. Adjust decision threshold to tune the trade-off.

Lectures 32–35: Advanced Algorithms & Validation

Support Vector Machine (SVM)

SVM finds the optimal hyperplane that maximises the margin between classes. The data points closest to the boundary are called support vectors.

Key Concept	Details
Hyperplane	Decision boundary: $w \cdot x + b = 0$
Margin	Distance between hyperplane and nearest points — SVM maximises this
Kernel trick	Maps data to higher dimensions to find a separable boundary. Kernels: linear, RBF, polynomial
C parameter	Regularisation — small C = wider margin + more misclassifications allowed; large C = narrow margin + fewer errors
Gamma (RBF)	Controls influence radius of each point. High gamma → complex boundary, overfitting risk
<pre>from sklearn.svm import SVC, SVR # Classification svc = SVC(kernel='rbf', C=1.0, gamma='scale') svc.fit(X_train, y_train) # Regression svr = SVR(kernel='rbf', C=100, epsilon=0.1) svr.fit(X_train, y_train)</pre>	

Decision Tree

Splits data into branches using feature thresholds to create a tree of decisions. Highly interpretable — you can visualise and follow the logic.

Key Concept	Details
Splitting criteria	Gini impurity (classification) or MSE (regression)
max_depth	Limit tree depth to prevent overfitting
min_samples_split	Minimum samples needed to split a node
Pruning	Remove branches that add little predictive power
Pros	Interpretable, handles mixed types, no scaling needed
Cons	Prone to overfitting (fixed by ensembles — see Random Forest)
<pre>from sklearn.tree import DecisionTreeClassifier, export_text dt = DecisionTreeClassifier(max_depth=4, min_samples_split=5, random_state=42) dt.fit(X_train, y_train) print(export_text(dt, feature_names=list(X.columns)))</pre>	

Train / Validation / Test Split & Cross-Validation

Set	Purpose Typical Size
Training set	Model learns from this → 60–70% of data
Validation set	Tune hyperparameters, compare models → 10–15%
Test set	Final evaluation — touch ONLY ONCE at the very end → 15–20%

Set	Purpose Typical Size
<pre># K-Fold Cross-Validation from sklearn.model_selection import cross_val_score, KFold kf = KFold(n_splits=5, shuffle=True, random_state=42) scores = cross_val_score(model, X, y, cv=kf, scoring='r2') print(f'CV R²: {scores.mean():.3f} ± {scores.std():.3f}')</pre> <pre># Stratified K-Fold (for classification – preserves class ratio) from sklearn.model_selection import StratifiedKFold skf = StratifiedKFold(n_splits=5)</pre>	

Bagging vs Boosting & Random Forest

Ensemble Method	How It Works Key Algorithm
Bagging	Train multiple models on random subsets of data (with replacement). Average predictions. Reduces variance. → Random Forest
Boosting	Train models sequentially — each corrects errors of the previous. Reduces bias. → XGBoost, AdaBoost, Gradient Boosting
<pre>from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier rf = RandomForestClassifier(n_estimators=100, max_depth=6, max_features='sqrt', random_state=42) rf.fit(X_train, y_train) # Feature importance import pandas as pd feat_imp = pd.Series(rf.feature_importances_, index=X.columns) feat_imp.sort_values(ascending=False).head(10)</pre>	

Overfitting, Underfitting, Bias & Variance

Concept	Definition Fix
Underfitting (High Bias)	Model too simple — misses patterns. Fix: more features, more complex model
Overfitting (High Variance)	Model memorises training data — poor on new data. Fix: regularisation, more data, pruning, ensemble
Bias-Variance Trade-off	Reducing bias often increases variance. Ensembles (RF, boosting) manage this balance.

Lectures 36–40: Unsupervised Learning & Clustering

Unsupervised Learning Overview

No labelled data. The goal is to discover hidden structure — groups, patterns, or anomalies — within the data.

Task	Algorithm Use Case
Clustering	K-Means, DBSCAN, Hierarchical — customer segments, document grouping
Dimensionality Reduction	PCA, t-SNE, UMAP — visualise high-dimensional data, compression
Anomaly Detection	Isolation Forest, One-Class SVM — fraud, network intrusion

Task	Algorithm Use Case
Association Rules	Apriori, FP-Growth — market basket analysis

K-Means Clustering

Partitions data into K clusters by minimising within-cluster variance. Iterates until centroids stabilise.

Step	Action
1. Initialise	Place K centroids randomly (or K-Means++ for smarter init)
2. Assign	Each point → nearest centroid (Euclidean distance)
3. Update	Recompute centroid = mean of all assigned points
4. Repeat	Steps 2–3 until centroids stop moving

```

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
import matplotlib.pyplot as plt

# Find optimal K using Elbow Method
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X)
    inertias.append(km.inertia_)

plt.plot(range(1,11), inertias, marker='o')
plt.xlabel('K'); plt.ylabel('Inertia'); plt.title('Elbow Method')
plt.show()

# Train final model
km = KMeans(n_clusters=3, random_state=42, n_init=10)
df['cluster'] = km.fit_predict(X)
print('Silhouette Score:', silhouette_score(X, df['cluster']))

```

Hierarchical Clustering

Builds a tree of clusters (dendrogram). No need to specify K in advance — you cut the dendrogram at a desired height.

Approach	Description
Agglomerative (bottom-up)	Start: each point is its own cluster. Merge closest pairs repeatedly.
Divisive (top-down)	Start: all points in one cluster. Split recursively.
Linkage methods	ward (minimise variance), complete (max distance), average, single

```

from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# Plot dendrogram
linked = linkage(X, method='ward')
dendrogram(linked, truncate_mode='level', p=5)
plt.title('Dendrogram'); plt.show()

# Fit model
hc = AgglomerativeClustering(n_clusters=3, linkage='ward')
df['cluster'] = hc.fit_predict(X)

```

DBSCAN Clustering

Density-Based Spatial Clustering. Groups together points that are closely packed. Automatically identifies outliers as noise. Does NOT require K.

Parameter	Meaning
eps	Maximum distance between two points to be neighbours
min_samples	Minimum points within eps radius to form a core point
Core point	Has $\geq$ min_samples neighbours within eps
Border point	In neighbourhood of a core point but has fewer than min_samples
Noise point	Neither core nor border — labelled as cluster = -1

```

from sklearn.cluster import DBSCAN

db = DBSCAN(eps=0.5, min_samples=5)
df['cluster'] = db.fit_predict(X)

n_clusters = len(set(df['cluster'])) - (1 if -1 in df['cluster'] else 0)
n_noise = list(df['cluster']).count(-1)
print(f'Clusters: {n_clusters}, Noise points: {n_noise}')

```

Anomaly Detection

Identifies rare observations that differ significantly from the majority. Used in fraud detection, network security, quality control.

Method	How It Works
Isolation Forest	Isolates anomalies by randomly splitting features — outliers require fewer splits
One-Class SVM	Learns a boundary around normal data; points outside = anomalies
Z-score method	Points > 3 standard deviations from mean = anomaly
IQR method	Points outside $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ = outlier

```

from sklearn.ensemble import IsolationForest

iso = IsolationForest(contamination=0.05, random_state=42)
# contamination = expected % of outliers in data

df['anomaly'] = iso.fit_predict(X)
# -1 = anomaly, 1 = normal

anomalies = df[df['anomaly'] == -1]
print(f'Detected {len(anomalies)} anomalies out of {len(df)} records')

```

End of Module 12 Notes — Generative AI Course